

PET ENGINEERING COLLEGE

An ISO 9001:2015 Certified Institution

Vallioor

Department of Artificial Intelligence and Datascience



CS3591-Computer Networks Laboratory

Reg No:

Name :

PET ENGINEERING COLLEGE

An ISO 9001:2015 Certified Institution

Vallioor



Certified that this is the bonafide record of work done by Mr/Ms.....of the fourth semester in **Artificial Intelligence and Datascience** of this college in the **CS3591- COMPUTER NETWORKS LABORATORY** during the year **2023-2024** in the partial fulfillment of the requirements of **B.Tech** course of the **ANNA UNIVERSITY,CHENNAI**.

Staff-in-Charge.

Mrs.S.Swathiga

AP/CSE

Head of the Department

Dr.S.Samsudeen Shaffi

HOD/AI/DS

University Reg.No.....

University Examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

CONTENTS

Sl. No	Date	Topic	Page	Sign
1.		LEARN TO USE COMMANDS LIKE TCPDUMP, NETSTAT, IFCONFIG, NSLOOKUP AND TRACEROUTE		
2.		WRITE A HTTP WEB CLIENT PROGRAM TO DOWNLOAD A WEB PAGE USING TCP		
3.		APPLICATIONS USING TCP SOCKETS LIKE: ECHO CLIENT AND ECHO SERVER, CHAT AND FILE TRANSFER		
4.		SIMULATION OF DNS USING UDP SOCKET		
5.		WRITE A CODE SIMULATING ARP /RARP PROTOCOLS		
6.		STUDY OF NETWORK SIMULATOR (NS) AND SIMULATION		
7.		STUDY OF TCP/UDP PERFORMANCE USING SIMULATION TOOL.		
8.		SIMULATION OF DISTANCE VECTOR/ LINK STATE ROUTING		

EX.NO:1

LEARN TO USE COMMANDS LIKE TCPDUMP, NETSTAT, IFCONFIG, NSLOOKUP AND TRACEROUTE. CAPTURE PING AND TRACEROUTE PDUS USING A NETWORK PROTOCOL ANALYZER AND EXAMINE.

AIM:

Commands:

Tcpdump:

Display traffic between 2 hosts:

To display all traffic between two hosts (represented by variables host1 and host2): # tcpdump host host1 and host2

Display traffic from a source or destination host only:

To display traffic from only a source (src) or destination (dst) host:

tcpdump src host

tcpdump dst host

Display traffic for a specific protocol

Provide the protocol as an argument to display only traffic for a specific protocol, for example tcp, udp, icmp, arp

```
# tcpdump protocol
```

For example to display traffic only for the tcp traffic :

```
# tcpdump tcp
```

Filtering based on source or destination port

To filter based on a source or destination port:

```
# tcpdump src port ftp
```

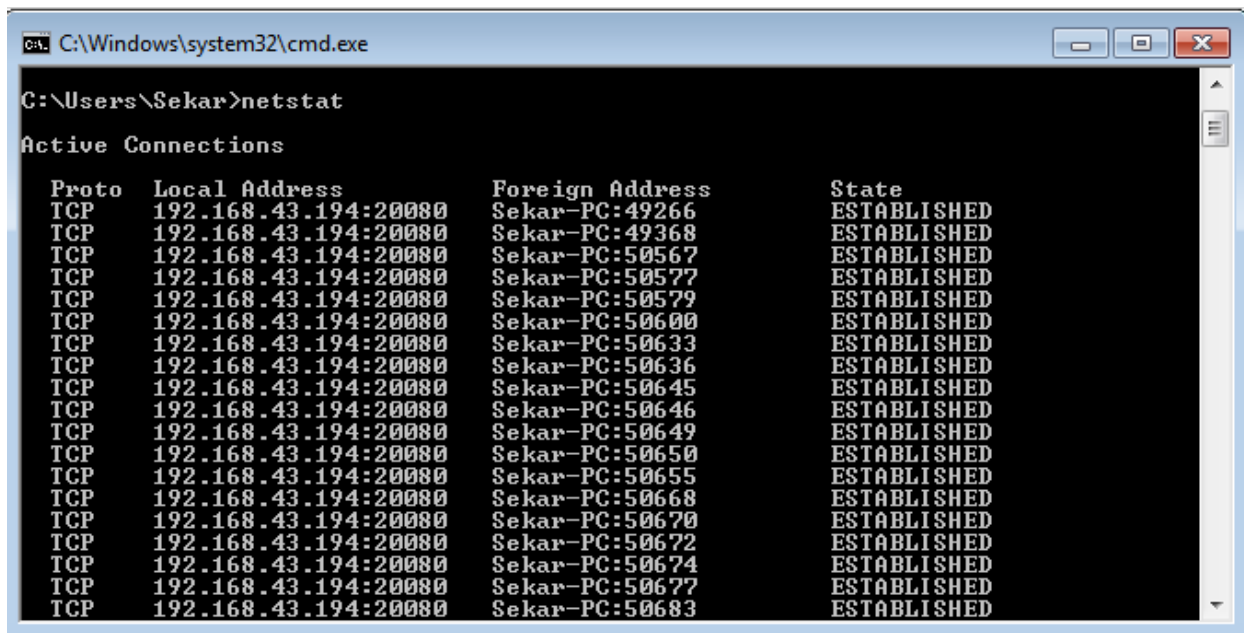
```
# tcpdump dst port http
```

2. Netstat

Netstat is a common command line TCP/IP networking available in most versions of Windows, Linux, UNIX and other operating systems.

Netstat provides information and statistics about protocols in use and current TCP/IP network connections. The Windows help screen (analogous to a Linux or UNIX for netstat reads as follows: displays protocol statistics and current TCP/IP network connections.

```
#netstat
```



```
C:\Windows\system32\cmd.exe
C:\Users\Sekar>netstat

Active Connections

Proto Local Address          Foreign Address         State
TCP   192.168.43.194:20080    Sekar-PC:49266         ESTABLISHED
TCP   192.168.43.194:20080    Sekar-PC:49368         ESTABLISHED
TCP   192.168.43.194:20080    Sekar-PC:50567         ESTABLISHED
TCP   192.168.43.194:20080    Sekar-PC:50577         ESTABLISHED
TCP   192.168.43.194:20080    Sekar-PC:50579         ESTABLISHED
TCP   192.168.43.194:20080    Sekar-PC:50600         ESTABLISHED
TCP   192.168.43.194:20080    Sekar-PC:50633         ESTABLISHED
TCP   192.168.43.194:20080    Sekar-PC:50636         ESTABLISHED
TCP   192.168.43.194:20080    Sekar-PC:50645         ESTABLISHED
TCP   192.168.43.194:20080    Sekar-PC:50646         ESTABLISHED
TCP   192.168.43.194:20080    Sekar-PC:50649         ESTABLISHED
TCP   192.168.43.194:20080    Sekar-PC:50650         ESTABLISHED
TCP   192.168.43.194:20080    Sekar-PC:50655         ESTABLISHED
TCP   192.168.43.194:20080    Sekar-PC:50668         ESTABLISHED
TCP   192.168.43.194:20080    Sekar-PC:50670         ESTABLISHED
TCP   192.168.43.194:20080    Sekar-PC:50672         ESTABLISHED
TCP   192.168.43.194:20080    Sekar-PC:50674         ESTABLISHED
TCP   192.168.43.194:20080    Sekar-PC:50677         ESTABLISHED
TCP   192.168.43.194:20080    Sekar-PC:50683         ESTABLISHED
```

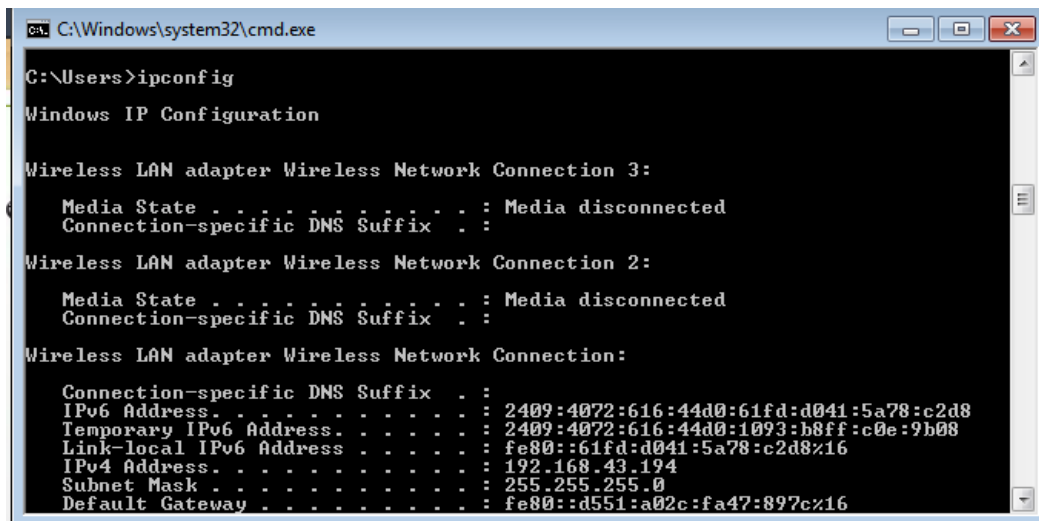
3. ipconfig

In Windows, **ipconfig** is a console application designed to run from the Windows command prompt. This utility allows you to get the IP address information of a Windows computer.

Using ipconfig

From the command prompt, type **ipconfig** to run the utility with default options. The output of the default command contains the IP address, network mask, and gateway for all physical and virtual network adapter.

```
#ipconfig
```



```
C:\Windows\system32\cmd.exe

C:\Users>ipconfig

Windows IP Configuration

Wireless LAN adapter Wireless Network Connection 3:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Wireless Network Connection 2:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Wireless Network Connection:

    Connection-specific DNS Suffix  . :
    IPv6 Address. . . . . : 2409:4072:616:44d0:61fd:d041:5a78:c2d8
    Temporary IPv6 Address. . . . . : 2409:4072:616:44d0:1093:b8ff:c0e:9b08
    Link-local IPv6 Address . . . . . : fe80::61fd:d041:5a78:c2d8%16
    IPv4 Address. . . . . : 192.168.43.194
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : fe80::d551:a02c:fa47:897c%16
```

4. nslookup

The **nslookup** (which stands for *name server lookup*) command is a network utility program used to obtain information about internet servers. It finds name server information for domains by querying the Domain Name System.

The nslookup command is a powerful tool for diagnosing DNS problems. You know you're experiencing a DNS problem when you can access a resource by specifying its IP address but not its DNS name.

```
#nslookup
```

5. Trace route:

Traceroute uses Internet Control Message Protocol (ICMP) echo packets with variable time to live (TTL) values. The response time of each hop is calculated. To guarantee accuracy, each hop is queried multiple times (usually three times) to better measure the response of that particular hop.

Traceroute is a network diagnostic tool used to track the pathway taken by a packet on an IP network from source to destination. Traceroute also records the time taken for each hop the packet makes during its route to the destination. Traceroute uses Internet Control Message Protocol (ICMP) echo packets with variable time to live (TTL) values.

The response time of each hop is calculated. To guarantee accuracy, each hop is queried multiple times (usually three times) to better measure the response of that particular hop. Traceroute sends packets with TTL values that gradually increase from packet to packet, starting with TTL value of one. Routers decrement TTL values of packets by one when routing and discard packets whose TTL value has reached zero, returning the ICMP error message ICMP Time Exceeded.

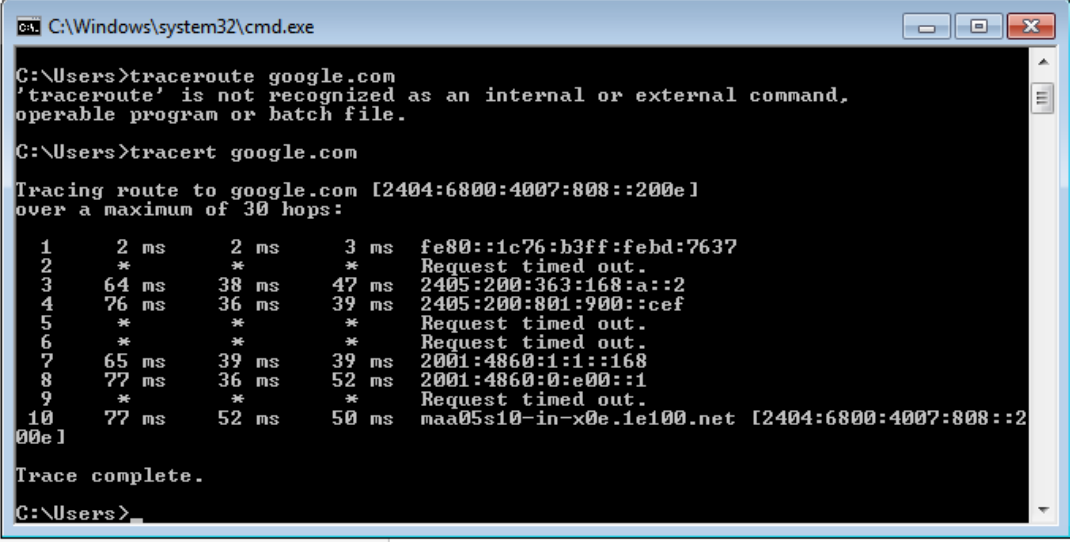
For the first set of packets, the first router receives the packet, decrements the TTL value and drops the packet because it then has TTL value zero. The router sends an ICMP Time Exceeded message back to the source. The next set of packets are given a TTL value of two, so the first router forwards the packets, but the second router drops them and replies with ICMP Time Exceeded.

Proceeding in this way, traceroute uses the returned ICMP Time Exceeded messages to build a list of routers that packets traverse, until the destination is reached and returns an ICMP Echo Reply message.

With the tracert command shown above, we're asking tracert to show us the path from the local computer all the way to the network device with the hostname

www.google.com.

#tracert google.com



```
C:\Windows\system32\cmd.exe

C:\Users>tracert google.com
'tracert' is not recognized as an internal or external command,
operable program or batch file.

C:\Users>tracert google.com

Tracing route to google.com [2404:6800:4007:808::200e]
over a maximum of 30 hops:

  0  2 ms  2 ms  3 ms  fe80::1c76:b3ff:febd:7637
  1  *      *      *      Request timed out.
  2  64 ms  38 ms  47 ms  2405:200:363:168:a::2
  3  76 ms  36 ms  39 ms  2405:200:801:900::cef
  4  *      *      *      Request timed out.
  5  *      *      *      Request timed out.
  6  65 ms  39 ms  39 ms  2001:4860:1:1::168
  7  77 ms  36 ms  52 ms  2001:4860:0:e00::1
  8  *      *      *      Request timed out.
  9  77 ms  52 ms  50 ms  maa05s10-in-x0e.1e100.net [2404:6800:4007:808::200e]

Trace complete.

C:\Users>
```

6. Ping:

The ping command sends an echo request to a host available on the network. Using this command, you can check if your remote host is responding well or not. Tracking and isolating hardware and software problems. Determining the status of the network and various foreign hosts. The ping command is usually used as a simple way to verify that a computer can communicate over the network with another computer or network device. The ping command operates by sending Internet Control Message Protocol (ICMP) Echo Request messages to the destination computer and waiting for a response

ping 172.16.6.2

```
C:\Windows\system32\cmd.exe

er).
    and has no effect on the type of service field in the IP Head
-r count    Record route for count hops <IPv4-only>.
-s count    Timestamp for count hops <IPv4-only>.
-j host-list Loose source route along host-list <IPv4-only>.
-k host-list Strict source route along host-list <IPv4-only>.
-w timeout  Timeout in milliseconds to wait for each reply.
-R          Use routing header to test reverse route also <IPv6-only>.
-S srcaddr  Source address to use.
-4          Force using IPv4.
-6          Force using IPv6.

C:\Users>ping 172.16.6.2

Pinging 172.16.6.2 with 32 bytes of data:
Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 172.16.6.2:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),

C:\Users>
```

RESULT

EX:2**WRITE A HTTP WEB CLIENT PROGRAM TO DOWNLOAD A WEB PAGE USING TCP****Ex.No.2 Write a HTTP web client program to download a web page using TCP sockets****Aim:**

To write a program in java to create a HTTP web client program to download a web page using TCP sockets.

Algorithm:

1. Start the program
2. Read the file to be downloaded from webpage
3. To download an image, use java URL class which can be found under java.net package.
4. The file is downloaded from server and is stored in the current working directory.
Stop the program

PROGRAM**Download.java**

```
import java.io.*;
import java.net.URL;
public class Download
{
    public static void main(String[] args) throws Exception
    {
        try
        {
            String fileName = "digital_image_processing.jpg";
            String website = "http://tutorialspoint.com/java_dip/images/"+fileName; System.out.println("Downloading File From: " + website);
            URL url = new URL(website);
            InputStream inputStream = url.openStream();
            OutputStream outputStream = new FileOutputStream(fileName); byte[] buffer = new byte[2048];
            int length = 0;
            while ((length = inputStream.read(buffer)) != -1)
            {
                System.out.println("Buffer Read of length: " + length); outputStream.write(buffer, 0, length);
            }
            inputStream.close(); outputStream.close();
        }
        catch(Exception e)
        {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

```
}  
}  
}
```

Output:

```
F:\Abarna\Lab>javac Download.java
```

```
F:\ Abarna \Lab>java Download
```

```
Downloading File From: http://tutorialspoint.com/java\_dip/images/digital\_image\_processing.jpg
```

```
Buffer Read of length: 2048
```

```
Buffer Read of length: 2048
```

```
Buffer Read of length: 2048
```

```
Buffer Read of length: 2048
```

```
Buffer Read of length: 2048
```

```
Buffer Read of length: 1097
```

```
Buffer Read of length: 2048
```

```
Buffer Read of length: 2048
```

```
Buffer Read of length: 1744
```

```
Buffer Read of length: 2048
```

```
Buffer Read of length: 2048
```

```
Buffer Read of length: 2048
```

```
Buffer Read of length: 2048
```

```
Buffer Read of length: 2048
```

```
Buffer Read of length: 1440
```

```
Buffer Read of length: 2048
```

```
Buffer Read of length: 20
```

Buffer Read of length: 2048Buffer
Read of length: 2048Buffer Read of
length: 2048Buffer Read of length:
2048Buffer Read of length: 2048
Buffer Read of length: 2048Buffer
Read of length: 2048Buffer Read of
length: 548 Buffer Read of length:
2048Buffer Read of length: 2048
Buffer Read of length: 2048Buffer
Read of length: 2048Buffer Read of
length: 2048Buffer Read of length:
2048Buffer Read of length: 2048
Buffer Read of length: 2048Buffer
Read of length: 1863
The downloaded file (Stored in the current working directory)



RESULT:

Ex.No: 3

**APPLICATIONS USING TCP SOCKETS LIKE: ECHO CLIENT AND ECHO SERVER,
CHAT AND FILE TRANSFER**

AIM

a. Echo client and echo server

ALGORITHM

CLIENT

1. Start
2. Create the TCP socket
3. Establish connection with the server
4. Get the message to be echoed from the user
5. Send the message to the server
6. Receive the message echoed by the server
7. Display the message received from the server
8. Terminate the connection
9. Stop

SERVER

1. Start
2. Create TCP socket, make it a listening socket
3. Accept the connection request sent by the client for connection establishment
4. Receive the message sent by the client
5. Display the received message
6. Send the received message to the client from which it receives
7. Close the connection when client initiates termination and server becomes a listening server, waiting for clients.
8. Stop.

PROGRAM:**EchoServer.java**

```
import java.net.*;
import java.io.*;
public class
EServer {
    public static void main(String
        args[]) {
        ServerSocket s=null;
        String line;
        DataInputStream is;
        PrintStream ps;
        Socket
        c=null;
        try
        {
            s=new ServerSocket(9000);
        } catch(IOException
        e) {
            System.out.println(e);
        }
        try
        {
            c=s.accept();
            is=new DataInputStream(c.getInputStream());
```

```

ps=new PrintStream(c.getOutputStream());while(true)
{
line=is.readLine();ps.println(line);
}
}
catch(IOException e)
{
System.out.println(e);
}
}
}

```

EClient.java

```

import java.net.*;
import java.io.*;
public class EClient
{
    public static void main(String
arg[])
    {
        Socket c=null;String line;
        DataInputStream is,is1;PrintStream os;
        try
        {
            InetAddress ia = InetAddress.getLocalHost();c=new
            Socket(ia,9000);
        } catch(IOException e) {
            System.out.println(e);
        }
        try
        {
            os=new PrintStream(c.getOutputStream());
            is=new DataInputStream(System.in); is1=new
            DataInputStream(c.getInputStream());
            while(true)
            {
                System.out.println("Client:");
                line=is.readLine();

```

```

        os.println(line);
        System.out.println("Server:" +
is1.readLine());
    }
} catch(IOException
e) {
    System.out.println("Socket
Closed!"); }
}}

```

OUTPUT

SERVER

```

C:\Program Files\Java\jdk1.5.0\bin>javac
EServer.java C:\Program Files\Java\jdk1.5.0\bin>java
EServer C:\Program Files\Java\jdk1.5.0\bin>

```

CLIENT

```

C:\Program Files\Java\jdk1.5.0\bin>javac
EClient.java C:\Program Files\Java\jdk1.5.0\bin>java
EClient Client: Hai Server
Server: Hai Server
Client: Hello
Server: Hello
Client: end
Server: end
Client: ds
Socket Closed!

```

B.Chat

AIM:

ALGORITHM

CLIENT

1. Start
2. Create the UDP datagram socket
3. Get the request message to be sent from the user
4. Send the request message to the server
5. If the request message is —END|| go to step 10
6. Wait for the reply message from the server
7. Receive the reply message sent by the server
8. Display the reply message received from the server

9. Repeat the steps from 3 to 8
10. Stop

ER

1. Start
2. Create UDP datagram socket, make it a listening socket
3. Receive the request message sent by the client
4. If the received message is —END— go to step 10
5. Retrieve the client's IP address from the request message received
6. Display the received message
7. Get the reply message from the user
8. Send the reply message to the client
9. Repeat the steps from 3 to 8.
10. Stop.

PROGRAM

UDPserver.java

```
import java.io.*;
import java.net.*;
class UDPserver
{
    public static DatagramSocket ds;
    public static byte buffer[]=new byte[1024];
    public static int clientport=789,serverport=790; public
    static void main(String args[])throws Exception {

        ds=new DatagramSocket(clientport);
        System.out.println("press ctrl+c to quit the program");
        BufferedReader dis=new BufferedReader(new InputStreamReader(System.in));
        InetAddress ia=InetAddress.getLocalHost();
        while(true)
        {
            DatagramPacket p=new DatagramPacket(buffer,buffer.length);ds.receive(p);
            String psx=new String(p.getData(),0,p.getLength());
            System.out.println("Client:" + psx); System.out.println("Server:");
            String str=dis.readLine();
            if(str.equals("end"))
                break; buffer=str.getBytes();
            ds.send(new DatagramPacket(buffer,str.length(),ia,serverport));
        }
    }
}
```

UDPclient.java

```
import java .io.*;
import java.net.*;
class UDPclient
{
public static DatagramSocket ds;
public static int clientport=789,serverport=790;
public static void main(String args[])throws Exception
{
byte buffer[]=new byte[1024]; ds=new
DatagramSocket(serverport);
BufferedReader dis=new BufferedReader(new InputStreamReader(System.in));
System.out.println("server waiting");
InetAddress ia=InetAddress.getLocalHost();while(true)
{
System.out.println("Client:");String
str=dis.readLine(); if(str.equals("end"))
break; buffer=str.getBytes();
ds.send(new DatagramPacket(buffer,str.length(),ia,clientport)); DatagramPacket
p=new DatagramPacket(buffer,buffer.length);ds.receive(p);
String psx=new String(p.getData(),0,p.getLength());
System.out.println("Server:" + psx);
}
}
}
```

OUTPUT:**SERVER**

```
C:\Program Files\Java\jdk1.5.0\bin>javac
UDPserver.java C:\Program Files\Java\jdk1.5.0\bin>java
UDPserver press ctrl+c to quit the program
Client:Hai Server
Server:Hello Client
Client:How are You
Server:I am Fine
```

CLIENT

C:\Program Files\Java\jdk1.5.0\bin>javac UDPclient.java

C:\Program Files\Java\jdk1.5.0\bin>java

UDPclient server waiting

Client:Hai Server

Server:Hello Client

Client:How are You

Server:I am Fine

Client:end

}

RESULT:

EX.NO: 4

SIMULATION OF DNS USING UDP SOCKET

AIM

ALGORITHM

SERVER

1. Start
2. Create UDP datagram socket
3. Create a table that maps host name and IP address
4. Receive the host name from the client
5. Retrieve the client's IP address from the received datagram
6. Get the IP address mapped for the host name from the table.
7. Display the host name and corresponding IP address
8. Send the IP address for the requested host name to the client
9. Stop.

CLIENT

1. Start
2. Create UDP datagram socket.
3. Get the host name from the client
4. Send the host name to the server
5. Wait for the reply from the server
6. Receive the reply datagram and read the IP address for the requested host name
7. Display the IP address.
8. Stop

PROGRAM

DNS SERVER

```
java import java.io.*;
import java.net.*;
public class
udpdnsserver {
    private static int indexOf(String[] array, String
                                str)
    {
        str = str.trim(); for (int i=0; i < array.length;
                                i++)
        return i;
    }

    {
        if (array[i].equals(str))
        return -1;
    }
    public static void main(String arg[]) throws
    IOException {
        String[] hosts = {"yahoo.com", "gmail.com", "cricinfo.com", "facebook.com"};
        String[] ip = {"68.180.206.184", "209.85.148.19", "80.168.92.140", "69.63.189.16"};
        System.out.println("Press Ctrl + C toQuit");
        while(true)
        {
            DatagramSocket serversocket = new DatagramSocket(1362);
            byte[] senddata = new byte[1021];
            byte[] receivedata = new byte[1021]; DatagramPacket recvpack = new DatagramPacket(receivedata,
            receivedata.length); serversocket.receive(recvpack);
            String sen = new String(recvpack.getData());
```

```

InetAddress ipaddress = recvpack.getAddress();
int port = recvpack.getPort();
String capsent;
System.out.println("Request for host " + sen);
if(indexOf (hosts, sen) != -1)
capsent = ip[indexOf (hosts, sen)];
else
capsent = "Host Not Found";
senddata = capsent.getBytes();
DatagramPacket pack = new DatagramPacket (senddata, senddata.length,ipaddress,port);serversocket.send(pack);
serversocket.close();
    }}}

```

UDP DNS Client

```

java import java.io.*;
import java.net.*;
public class udpdnsclient
{
    public static void main(String args[])throws IOException
    {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        DatagramSocket clientsocket = new DatagramSocket();
        InetAddress ipaddress;if
        (args.length == 0)
        ipaddress = InetAddress.getLocalHost();
        else

                ipaddress = InetAddress.getByName(args[0]);
                byte[] senddata = new byte[1024];
                byte[] receivedata = new byte[1024];int portaddr = 1362;
                System.out.print("Enter the hostname : ");
                String sentence = br.readLine();
                Senddata = sentence.getBytes();
                DatagramPacket pack = new DatagramPacket(senddata,senddata.length,
                clientsocket.send(pack);

                DatagramPacket recvpack =new DatagramPacket(receivedata,receivedata.length);
                clientsocket.receive(recvpack);
                String modified = new String(recvpack.getData());
                System.out.println("IP Address: " + modified); clientsocket.close();

ipaddress,portad
dr);

```

OUTPUT

SERVER

javac udpdnsserver.java

java udpdnsserver

Press Ctrl + C to Quit Request for host yahoo.com

Request for host cricinfo.com

Request for host youtube.com

CLIENT

>javac udpdnsclient.java

>java udpdnsclient

Enter the hostname : yahoo.com

IP Address: 68.180.206.184

>java udpdnsclient

Enter the hostname : cricinfo.com

IP Address: 80.168.92.140

>java udpdnsclient

Enter the hostname : youtube.com

IP Address: Host Not Found

RESULT

EX.NO:5

WRITE A CODE SIMULATING ARP /RARP PROTOCOLS

AIM:

a)Program for Address Resolution Protocol (ARP) using UDP

ALGORITHM:

CLIENT

1. Start the program
2. Create socket and establish connection with the server.
3. Get the IP address to be converted into MAC address from the user.
4. Send this IP address to server.
5. Receive the MAC address for the IP address from the server.
6. Display the received MAC address
7. Terminate the connection

SERVER

1. Start the program
2. Create the socket, bind the socket created with IP address and port number and make it a listening socket.
3. Accept the connection request when it is requested by the client.
4. Server maintains the table in which IP and corresponding MAC addresses are stored.
5. Receive the IP address sent by the client.
6. Retrieve the corresponding MAC address for the IP address and send it to the client.
7. Close the connection with the client and now the server becomes a listening server waiting for the connection request from other clients
8. Stop

PROGRAM

CLIENT:

```
import java.io.*;
import java.net.*;
import java.util.*;
class Clientarp
{
```



```

public static void main(String
args[]) {
try
{
    BufferedReader in=new BufferedReader(new InputStreamReader(System.in));
    Socket clsct=new Socket("127.0.0.1",139)
    DataInputStream din=new DataInputStream(clsct.getInputStream());
    DataOutputStream dout=new DataOutputStream(clsct.getOutputStream());
    System.out.println("Enter the Logical address(IP):");
    String str1=in.readLine();
    dout.writeBytes(str1+'\n');
    String str=din.readLine();

        System.out.println("The Physical Address is: "+str);
        clsct.close();
    }
    catch (Exception e)
    {
        System.out.println(e);
    }
}

```

SERVER:

```

import java.io.*;
import java.net.*;
import java.util.*;

class Serverarp
{
public static void main(String args[])
{
try{
ServerSocket obj=new
ServerSocket(139);
Socketobj1=obj.accept(); while(true)
{
DataInputStream din=new DataInputStream(obj1.getInputStream());
DataOutputStream dout=new
DataOutputStream(obj1.getOutputStream());String str=din.readLine();
String ip[]={"165.165.80.80","165.165.79.1"};
String mac[]={"6A:08:AA:C2","8A:BC:E3:FA"};

```

```
for(int i=0;i<ip.length;i++)  
  
if(str.equals(ip[i]))  
{  
dout.writeBytes(mac[i]+'\\n');  
  
    break;  
}  
}  
obj.close();  
}  
}  
catch(Exception e)  
{  
    System.out.println(e);  
}}  
}
```

OUTPUT:

E:\networks>java Serverarp

E:\networks>java Clientarp

Enter the Logical address(IP):

165.165.80.80

The Physical Address is: 6A:08:AA:C2

(b) Program for Reverse Address Resolution Protocol (RARP) using UDP

ALGORITHM:

CLIENT

1. Start the program
2. Create datagram socket
3. Get the MAC address to be converted into IP address from the user.
4. Send this MAC address to server using UDP datagram.
5. Receive the datagram from the server and display the corresponding IP address.
6. Stop

SERVER

1. Start the program.
2. Server maintains the table in which IP and corresponding MAC addresses are stored.
3. Create the datagram socket
4. Receive the datagram sent by the client and read the MAC address sent.
5. Retrieve the IP address for the received MAC address from the table.
6. Display the corresponding IP address.
7. Stop

PROGRAM:

CLIENT:

```
import java.io.*;
import java.net.*;
import java.util.*;
class
Clientarp12 {
    public static void main(String args[])
    {
        DatagramSocket client=new DatagramSocket();
        InetAddress addr=InetAddress.getByName("127.0.0.1");
        byte[] sendbyte=new byte[1024];
        byte[] receivebyte=new byte[1024];
        BufferedReader in=new BufferedReader(new InputStreamReader(System.in));
        System.out.println("Enter the Physical address (MAC):")
        String str=in.readLine(); sendbyte=str.getBytes();
        DatagramPacket sender=new DatagramPacket(sendbyte,sendbyte.length,addr,1309);
        client.send(sender);
        DatagramPacket receiver=new DatagramPacket(receivebyte,receivebyte.length);
        client.receive(receiver);
```

```
String s=new String(receiver.getData());

System.out.println("The Logical Address is(IP): "+s.trim());client.close();
}
catch(Exception e)
{
System.out.println(e);
}}}
```

Server:

```
import java.io.*;
import java.net.*;
import java.util.*;
class Serverrarp12
{
public static void main(String args[])
{
try{
    DatagramSocket server=new DatagramSocket(1309);
    while(true)
    {
byte[] sendbyte=new byte[1024];
byte[] receivebyte=new byte[1024];
DatagramPacket receiver=new DatagramPacket(receivebyte,receivebyte.length);
server.receive(receiver);
String str=new String(receiver.getData());String s=str.trim();
InetAddress addr=receiver.getAddress();int port=receiver.getPort();
String ip[]={"165.165.80.80","165.165.79.1"};
String mac[]={ "6A:08:AA:C2","8A:BC:E3:FA"};
for(int i=0;i<ip.length;i++)
{
if(s.equals(mac[i]))
{
                sendbyte=ip[i].getBytes();
                DatagramPacket sender = new
                DatagramPacket(sendbyte,sendbyte.length,addr,port);
                server.send(sender);
                break;
            }
        }
    }
break;
}
```

```
}  
}  
}catch(Exception e)  
{  
System.out.println(e);  
}  
}  
}
```

OUTPUT:

```
I:\ex>java Serverarp12  
I:\ex>java Clientarp12  
Enter the Physical address (MAC):  
6A:08:AA:C2  
The Logical Address is(IP): 165.165.80.80  
.
```

RESULT :

Ex.No: 6

STUDY OF NETWORK SIMULATOR (NS) AND SIMULATION OF CONGESTION CONTROL ALGORITHMS USING NS

AIM:

NET WORK SIMULATOR (NS2)

Ns Overview

- Ns Status
- Periodical release (ns-2.26, Feb 2003)
- Platform support
- FreeBSD, Linux, Solaris, Windows and Mac

Ns

functionalities

Routing, Transportation, Traffic sources, Queuing disciplines, QoS

Congestion Control Algorithms

- Slow start
- Additive increase/multiplicative decrease
- Fast retransmit and Fast recovery

Case Study: A simple Wireless network.

Ad hoc routing, mobile IP, sensor-MAC Tracing, visualization and various utilities NS (Network Simulators) Most of the commercial simulators are GUI driven, while some network simulators are CLI driven. The network model / configuration describes the state of the network (nodes, routers, switches, links) and the events (data transmissions, packet error etc.). An important output of simulations are the trace files. Trace files log every packet, every event that occurred in the simulation and are used for analysis. Network simulators can also provide other tools to facilitate visual analysis of trends and potential trouble spots.

Most network simulators use discrete event simulation, in which a list of pending "events" is stored, and those events are processed in order, with some events triggering future events—such as the event of the arrival of a packet at one node triggering the event of the arrival of that packet at a downstream node.

Simulation of networks is a very complex task. For example, if congestion is high, then estimation of the average occupancy is challenging because of high variance. To

estimate the likelihood of a buffer overflow in a network, the time required for an accurate answer can be extremely large. Specialized techniques such as "control variates" and "importance sampling" have been developed to speed simulation. Examples of network simulators

There are many both free/open-source and proprietary network simulators. Examples of notable network simulation software are, ordered after how often they are mentioned in research papers:

1. ns (open source)
2. OPNET (proprietary software)
3. 3.NetSim (proprietary software)

PROGRAM:

```
include <wifi_lte/wifi_lte_rtable.h>struct r_hist_entry *elm, *elm2;
int num_later = 1;
elm = STAILQ_FIRST(&r_hist_);
while (elm != NULL && num_later <= num_dup_acks_)
{
    num_later;
    elm = STAILQ_NEXT(elm, linfo_);
}
if (elm != NULL){
    elm = findDataPacketInRecvHistory(STAILQ_NEXT(elm,linfo_));
    if (elm != NULL){
        elm2 = STAILQ_NEXT(elm, linfo_);while(elm2 != NULL)
        {
            if (elm2->seq_num_ < seq_num && elm2->t_recv_ <
time){
                STAILQ_REMOVE(&r_hist_,elm2,r_hist_entry,linfo_);
                delete elm2;
            } else
            elm = elm2;
            elm2 = STAILQ_NEXT(elm, linfo_);
        }
    }
}
void DCCPTFRCAgent::removeAcksRecvHistory()
{
    struct r_hist_entry *elm1 = STAILQ_FIRST(&r_hist_);
    struct r_hist_entry *elm2;

    int num_later = 1;
    while (elm1 != NULL && num_later <= num_dup_acks_)
```

```

{
    num_later;
    elm1 = STAILQ_NEXT(elm1, linfo_);
}
if(elm1 == NULL)
return;
elm2 = STAILQ_NEXT(elm1, linfo_);while(elm2 != NULL)
{
if (elm2->type_ == DCCP_ACK)
{
STAILQ_REMOVE(&r_hist_,elm2,r_hist_entry,linfo_);
delete elm2;
} else {
elm1 = elm2;
}
elm2 = STAILQ_NEXT(elm1, linfo_);
}
}
inline r_hist_entry
*DCCPTFRCAgent::findDataPacketInRecvHistory(r_hist_entry *start)
{
while(start != NULL && start->type_ == DCCP_ACK)
start = STAILQ_NEXT(start,linfo_);
return start;
}

```

RESULT

Ex.No: 7

STUDY OF TCP/UDP PERFORMANCE USING SIMULATION TOOL.

AIM:

PRE LAB DISCUSSION:

- TCP is reliable protocol. That is, the receiver always sends either positive or negative acknowledgement about the data packet to the sender, so that the sender always has bright clue about whether the data packet is reached the destination or it needs to resend it.
- TCP ensures that the data reaches intended destination in the same order it was sent.
- TCP is connection oriented. TCP requires that connection between two remote points be established before sending actual data.
- TCP provides error-checking and recovery mechanism.
- TCP provides end-to-end communication.
- TCP provides flow control and quality of service.
- TCP operates in Client/Server point-to-point mode.
- TCP provides full duplex server, i.e. it can perform roles of both receiver and sender.
- The User Datagram Protocol (UDP) is simplest Transport Layer communication protocol available of the TCP/IP protocol suite. It involves minimum amount of communication mechanism. UDP is said to be an unreliable transport protocol but it uses IP services which provides best effort delivery mechanism.UDP is used when acknowledgement of data does not hold any significance.
- UDP is good protocol for data flowing in one direction.
- UDP is simple and suitable for query based communications.
- UDP is not connection oriented.
- UDP does not provide congestion control mechanism.
- UDP does not guarantee ordered delivery of data.
- UDP is stateless.
- UDP is suitable protocol for streaming applications such as VoIP, multimedia streaming.

TCP Performance

Algorithm

1. Create a Simulator object.
2. Set routing as dynamic.
3. Open the trace and nam trace files.
4. Define the finish procedure.
5. Create nodes and the links between them.
6. Create the agents and attach them to the nodes.

7. Create the applications and attach them to the tcp agent. Connect tcp and tcp sink.
8. Run the simulation.

PROGRAM:

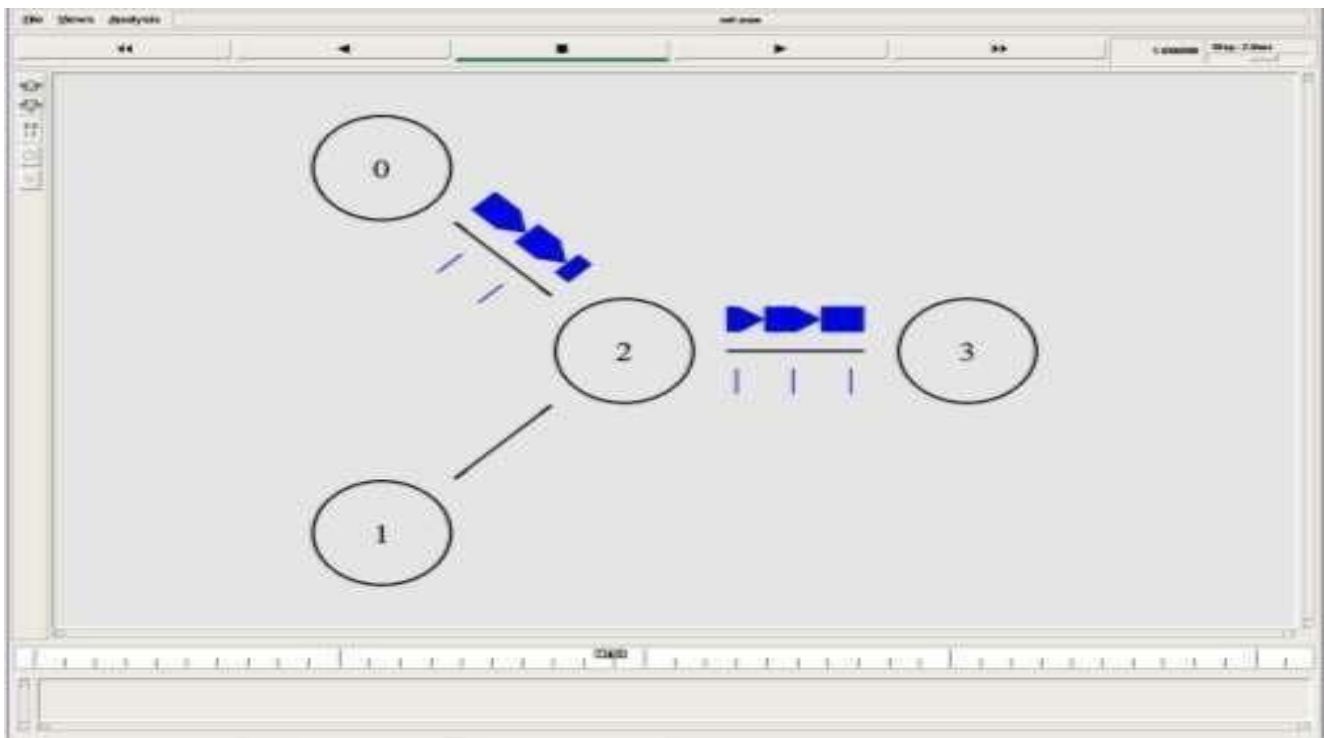
```
set ns [new Simulator]
$ns color 0 Blue
$ns color 1 Red
$ns color 2
Yellow set n0
[$ns node] set
n1[$ns node]
set n2[$ns
node] set n3
[$ns node]
set f [open tcpout.tr w]
$ns trace-all $f
set nf [open tcpout.nam w]
$ns namtrace-all $nf
$ns duplex-link $n0 $n2 5Mb 2ms DropTail
$ns duplex-link $n1 $n2 5Mb 2ms DropTail
$ns duplex-link $n2 $n3 1.5Mb 10ms DropTail
$ns duplex-link-op $n0 $n2 orient right-up
$ns duplex-link-op $n1 $n2 orient right-down
$ns duplex-link-op $n2 $n3 orient right
$ns duplex-link-op $n2 $n3 queuePos
0.5 set tcp [new Agent/TCP]
$tcp set class_ 1
set sink [new Agent/TCPSink]
$ns attach-agent $n1 $tcp
$ns attach-agent $n3 $sink
$ns connect $tcp $sink
```

```

set ftp [new Application/FTP]
$ftp attach-agent $tcp
$ns at 1.2 "$ftp start"
$ns at 1.35 "$ns detach-agent $n1 $tcp ; $ns detach-agent $n3 $sink"
$ns at 3.0 "finish"
proc finish {} {
    global ns f nf
$ns flush- trace
close $fclose $nf
puts "Running nam.."
exec xgraph tcpout.tr -geometry 600x800 &
exec namtcpout.nam &
exit 0
}
$ns run

```

OUTPUT



UDP Performance

ALGORITHM :

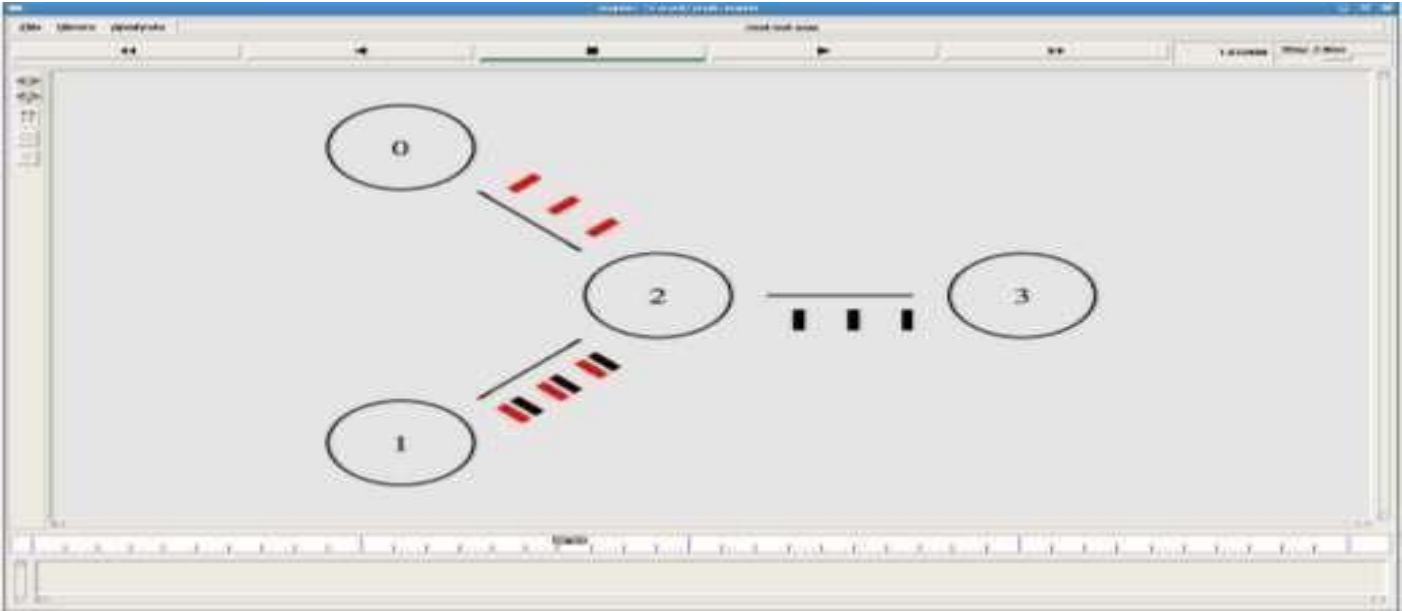
1. Create a Simulator object.
2. Set routing as dynamic.
3. Open the trace and nam trace files.
4. Define the finish procedure.
5. Create nodes and the links between them.
6. Create the agents and attach them to the nodes.
7. Create the applications and attach them to the UDP agent.
8. Connect udp and null agents.
9. Run the simulation.

PROGRAM:

```
set ns [new Simulator]
$ns color 0 Blue
$ns color 1 Red
$ns color 2
Yellow set n0
[$ns node] set n1
[$ns node] set n2
[$ns node] set n3
[$ns node]
set f [open udpout.tr w]
$ns trace-all $f
set nf [open udpout.nam w]
$ns namtrace-all $nf
$ns duplex-link $n0 $n2 5Mb 2ms DropTail
$ns duplex-link $n1 $n2 5Mb 2ms DropTail
$ns duplex-link $n2 $n3 1.5Mb 10ms DropTail
$ns duplex-link-op $n0 $n2 orient right-up
$ns duplex-link-op $n1 $n2 orient right-down
$ns duplex-link-op $n2 $n3 orient right
$ns duplex-link-op $n2 $n3 queuePos
0.5 set udp0 [new Agent/UDP]
$ns attach-agent $n0 $udp0
set cbr0 [new Application/Traffic/CBR]
$cbr0 attach-agent $udp0
set udp1 [new
```

```
Agent/UDP]
$ns attach-agent $n3 $udp1
$udp1 set class_ 0
set cbr1 [new Application/Traffic/CBR]
$cbr1 attach-agent $udp1
setnull0 [new
Agent/Null]
$ns attach-agent $n1
$null0 set null1
[newAgent/Null]
$ns attach-agent $n1 $null1
$ns connect $udp0 $null0
$ns connect $udp1 $null1
$ns at 1.0 "$cbr0 start"
$ns at 1.1 "$cbr1 start"
puts [$cbr0 set
packetSize_] puts [$cbr0
set interval_]
$ns at 3.0 "finish"
procfinish {} {
global ns f nf
$ns flush- trace close
$fclose $nf
puts "Running nam.." exec nam
udpout.nam & exit 0
}
$ns run
```

OUTPUT



RESULT :

Ex.No: 8

SIMULATION OF DISTANCE VECTOR/ LINK STATE ROUTING ALGORITHM

AIM:

ALGORITHM:

1. Create a Simulator object.
2. Set routing as dynamic.
3. Open the trace and nam trace files.
4. Define the finish procedure.
5. Create nodes and the links between them.
6. Create the agents and attach them to the nodes.
7. Create the applications and attach them to the udp agent.
8. Connect udp and null..
9. At 1 sec the link between node 1 and 2 is broken.
10. At 2 sec the link is up again.
11. Run the simulation.

LINK STATE ROUTING PROTOCOL

PROGRAM

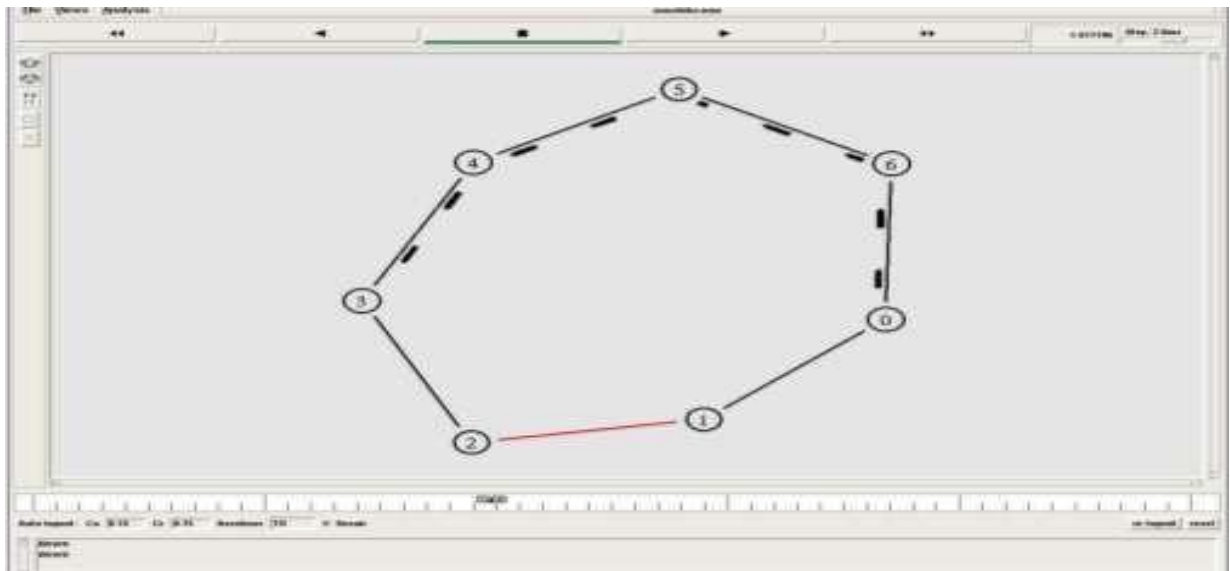
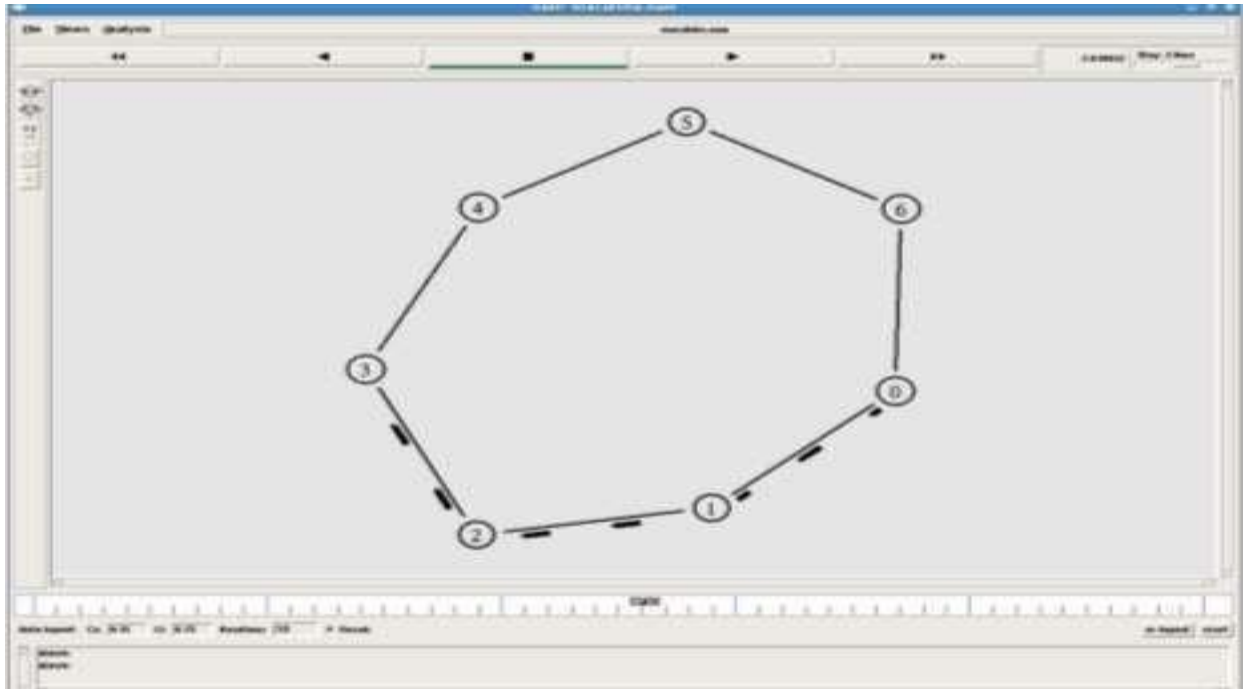
```
set ns [new Simulator]
$ns rtproto LS
set nf [open linkstate.nam w]
$ns namtrace-all $nf
set f0 [open linkstate.tr w]
$ns trace-all
$f0 proc finish
{} {
global ns f0 nf
$ns flush-trace close
$f0close $nf
exec nam linkstate.nam & exit 0
}
```

```
for {set i 0} {$i < 7} {incr i}
{ set n($i) [$ns node]
}
for {set i 0} {$i < 7} {incr i} {
$ns duplex-link $n($i) $n([expr ($i+1)%7]) 1Mb 10ms DropTail
}
set udp0 [new Agent/UDP]
$ns attach-agent $n(0) $udp0
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0
set null0 [new
Agent/Null]
$ns attach-agent $n(3) $null0
$ns connect $udp0 $null0
$ns at 0.5 "$cbr0 start"
$ns rtmodel-at 1.0 down $n(1) $n(2)
$ns rtmodel-at 2.0 up $n(1) $n(2)
```


\$ns at 4.5 "\$cbr0 stop"

\$ns at 5.0 "finish"

\$ns run



DISTANCE VECTOR ROUTING ALGORITHM

ALGORITHM:

1. Create a simulator object
2. Set routing protocol to Distance Vector routing
3. Trace packets on all links onto NAM trace and text trace file
4. Define finish procedure to close files, flush tracing and run NAM
5. Create eight nodes
6. Specify the link characteristics between nodes
7. Describe their layout topology as a octagon
8. Add UDP agent for node n1
9. Create CBR traffic on top of UDP and set traffic parameters.
10. Add a sink agent to node n4
11. Connect source and the sink
12. Schedule events as follows:
 - a. Start traffic flow at 0.5
 - b. Down the link n3-n4 at 1.0
 - c. Up the link n3-n4 at 2.0
 - d. Stop traffic at 3.0
 - e. Call finish procedure at 5.0
13. Start the scheduler
14. Observe the traffic route when link is up and down
15. View the simulated events and trace file analyze it
16. Stop

PROGRAM

```
#Distance vector routing protocol –
distvect.tcl #Create a simulator object
set ns [new Simulator]
#Use distance vector routing
$ns rtproto DV
#Open the nam trace
file set nf [open
out.nam w]
$ns namtrace-all
$nf # Open
tracefile
```

```

set nt [open trace.tr w]
$ns trace-all $nt
#Define 'finish' procedure

```

```

proc finish {}
{
global ns nf
$ns flush-trace #Close the
trace file close $nf
#Execute nam on the trace file exec nam -a
out.nam &
exit 0
}

# Create 8 nodes set n1 [$ns node] set n2 [$ns node] set n3 [$ns node] set n4 [$ns node] set n5 [$ns node] set
n6 [$ns node] set n7 [$ns node] set n8 [$ns node]
# Specify link characteristics
$ns duplex-link $n1 $n2 1Mb 10ms DropTail
$ns duplex-link $n2 $n3 1Mb 10ms DropTail
$ns duplex-link $n3 $n4 1Mb 10ms DropTail
$ns duplex-link $n4 $n5 1Mb 10ms DropTail
$ns duplex-link $n5 $n6 1Mb 10ms DropTail
$ns duplex-link $n6 $n7 1Mb 10ms DropTail
$ns duplex-link $n7 $n8 1Mb 10ms DropTail
$ns duplex-link $n8 $n1 1Mb 10ms DropTail #
specify layout as a octagon
$ns duplex-link-op $n1 $n2 orient left-up
$ns duplex-link-op $n2 $n3 orient up
$ns duplex-link-op $n3 $n4 orient right-up
$ns duplex-link-op $n4 $n5 orient right
$ns duplex-link-op $n5 $n6 orient right-down
$ns duplex-link-op $n6 $n7 orient down
$ns duplex-link-op $n7 $n8 orient left-down
$ns duplex-link-op $n8 $n1 orient left
#Create a UDP agent and attach it to node n1
set udp0 [new Agent/UDP]
$ns attach-agent $n1 $udp0
#Create a CBR traffic source and attach it to udp0
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500

```

```

$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0
#Create a Null agent (a traffic sink) and attach it to node n4
set null0 [new Agent/Null]
$ns attach-agent $n4 $null0
#Connect the traffic source with the traffic sink
$ns connect $udp0 $null0

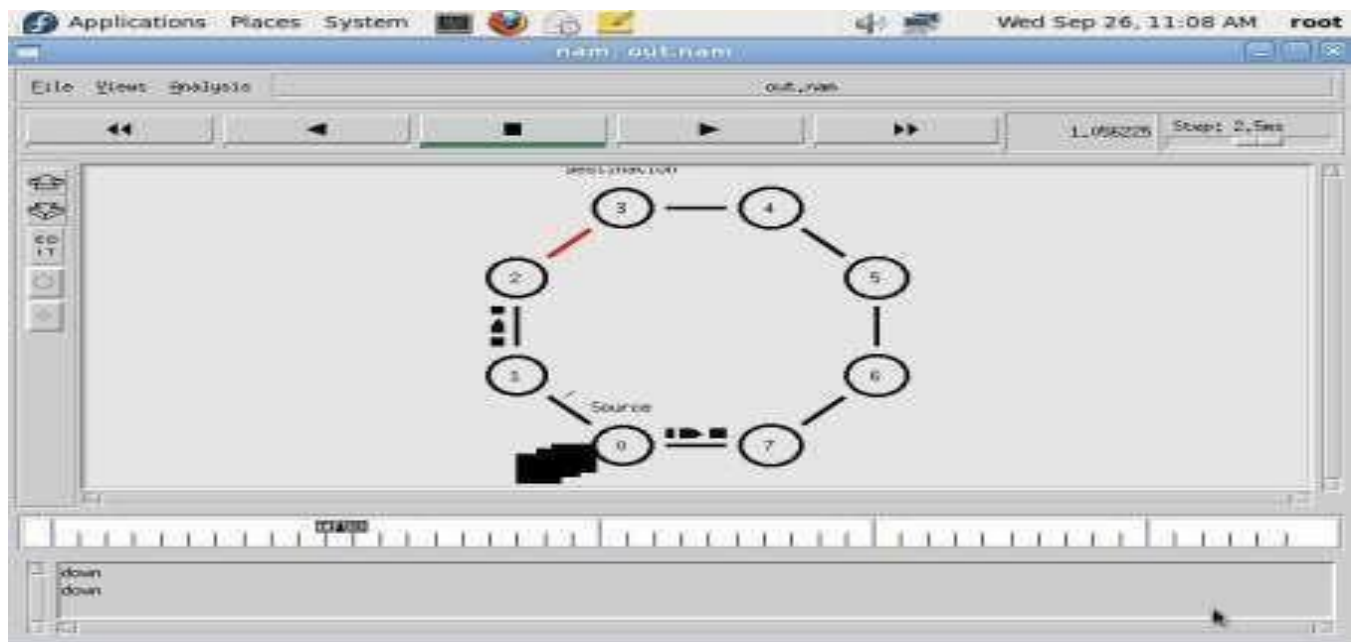
#Schedule events for the CBR agent and the network dynamics
$ns at 0.0 "$n1 label Source"
$ns at 0.0 "$n4 label Destination"
$ns at 0.5 "$cbr0 start"
$ns rtmodel-at 1.0 down $n3 $n4
$ns rtmodel-at 2.0 up $n3 $n4
$ns at 4.5 "$cbr0 stop"

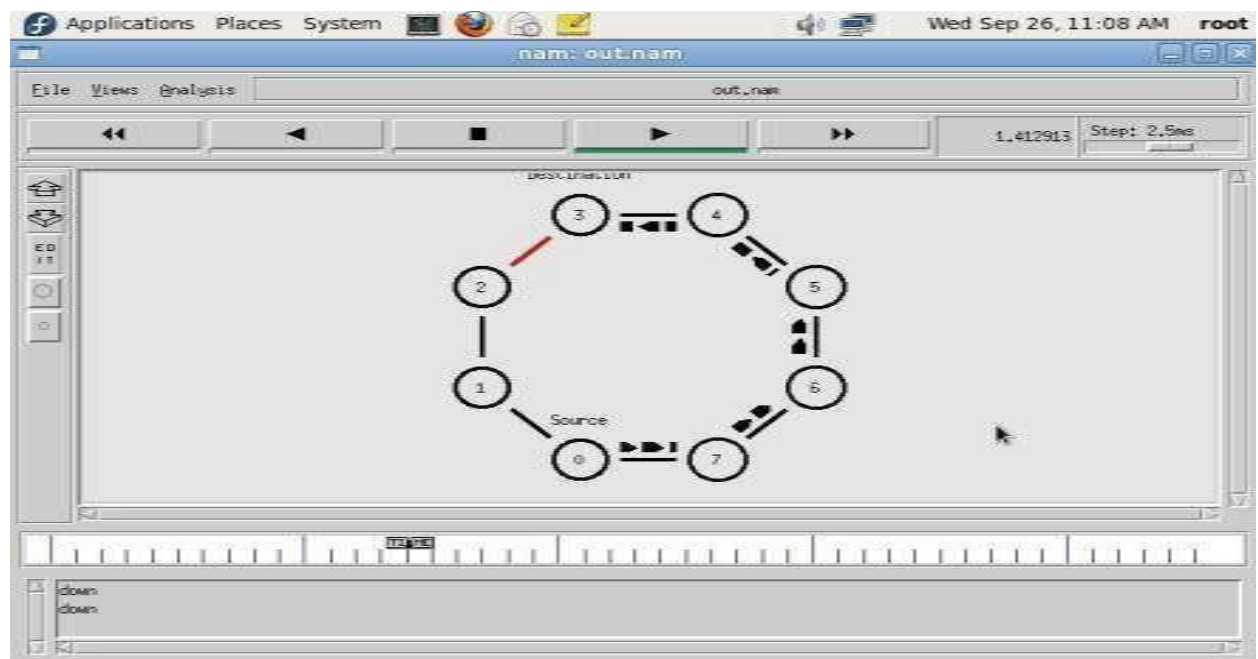
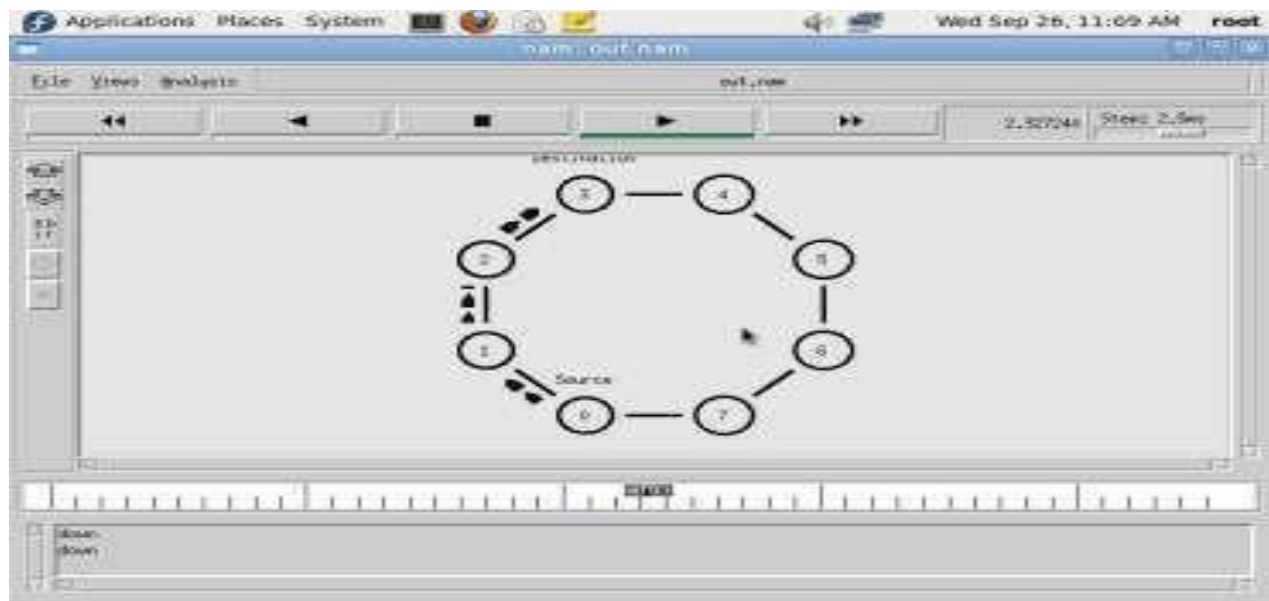
#Call the finish procedure after 5 seconds of simulation time
$ns at 5.0 "finish" #Run
the simulation
$ns run

```

OUTPUT

`$ ns distvect.tcl`





RESULT: